

DATABASE PORTFOLIO: PROJECT 3

HEALTHPULSE MEDICAL SYSTEMS NETWORK

1. The Scene & Business Objective

The Scenario: HealthPulse coordinates a multi-center healthcare clinic network handling critical patient directory tracking, clinical department appointments, and complex insurance financial claims.

The Operational Challenges:

- **Demographic Care Auditing:** Clinical executives must identify high-risk age demographics (patients over 40) who exhibit repeating appointment volume patterns to distribute hospital staffing correctly.
- **Insurance Payout Discrepancies:** Medical financial audits require tracking insurance provider claims against historical internal baseline med-bill costs, isolating high-value insurance invoices while omitting self-pay cash profiles.
- **Front-Desk Search Concurrency:** Real-time clinical lookup tools demand ultra-fast lookups when query engines crawl through millions of records looking for patient appointment histories.

Objective: To engineer a custom database schema entirely from scratch, implementing structural constraints, multi-table relationship links, aggregate reporting procedures, and performance optimizations.

2. Simplified Tasks, Queries, and Technical Outputs

Task 1: High-Risk Patient Activity Audit

Goal: Extract all patient profiles where age exceeds 40, calculating their historical treatment volumes using advanced grouping mechanics.

Query Implemented:

```
WITH CTE_Total_Visits_Above_AGE40 AS (  
    SELECT Patients.PatientID, FullName, Age, COUNT(VisitID) AS Total_Visits  
    FROM Patients  
    JOIN Visits ON Patients.PatientID = Visits.PatientID  
    WHERE Age > 40  
    GROUP BY Patients.PatientID, FullName, Age  
)  
SELECT * FROM CTE_Total_Visits_Above_AGE40  
ORDER BY Total_Visits DESC;
```

Output Dataset:

PatientID	FullName	Age	Total_Visits
P-101	Babajide Alao	45	2
P-103	Aminu Musa	62	2
P-106	Zainab Umar	51	1

Task 2: High-Value Insurance Leak Audit

Goal: Form a multi-level 3-table data chain linking demographics, visits, and billing ledgers together. Isolate non-cash insurance claims exceeding the network average baseline (\$1,625.00).

Query Implemented:

```
CREATE VIEW V_high_value_insurance_transactions AS
SELECT
    FullName,
    VisitDate,
    Department,
    Insurance_Provider,
    Cost_USD
FROM Patients
JOIN Visits ON Patients.PatientID = Visits.PatientID
JOIN Billing ON Visits.VisitID = Billing.VisitID
WHERE Insurance_Provider <> 'Cash';
GO

SELECT * FROM V_high_value_insurance_transactions
WHERE Cost_USD > (SELECT AVG(Cost_USD) FROM V_high_value_insurance_transactions);
```

Output Dataset:

FullName	VisitDate	Department	Insurance_Provider	Cost_USD
Zainab Umar	2026-05-03	Oncology	Leadway Health	5200.00

Task 3: Departmental Revenue Performance Engine

Goal: Provide administrative hospital boards with an automated on-demand script to audit incoming department revenue totals and attendance volumes synchronously.

Query Implemented:

```
CREATE PROCEDURE sp_GetDepartmentFinancials
    @TargetDept VARCHAR(50) = NULL
AS
SELECT
    Department,
    COUNT(Visits.VisitID) AS Total_Visits,
    SUM(Cost_USD) AS Total_Revenue
FROM Visits
JOIN Billing ON Visits.VisitID = Billing.VisitID
WHERE Department = @TargetDept OR @TargetDept IS NULL
GROUP BY Department
ORDER BY Total_Revenue DESC;
GO
```

Testing the Engine:

```
EXEC sp_GetDepartmentFinancials @TargetDept = 'Cardiology';
```

Output Dataset:

Department	Total_Visits	Total_Revenue
Cardiology	3	3750.00

Task 4: High-Velocity Medical Record Indexing

Goal: Construct physical data mapping infrastructure on the core activity ledger table to eliminate query lag during patient search interactions.

Query Implemented:

```
CREATE INDEX IX_Visits_Patient_Date
ON Visits (PatientID, VisitDate);
```

3. Conclusion & Core Competencies Shown

This project proves complete foundational database engineering capabilities:

- **Relational DDL Design:** Writing schema scripts from scratch utilizing PRIMARY KEY, auto-incrementing IDENTITY boundaries, and explicit FOREIGN KEY REFERENCES constraints.
- **Multi-Table Mapping Mastery:** Stacking multi-level inner joins across a 3-table relational chain without causing structural cross-product errors.
- **Modular Code Refactoring:** Simplifying multi-table join filters into clear Virtual Views for rapid subquery injection.